



K.L.E. Society's
K. L. E. INSTITUTE OF TECHNOLOGY, HUBBALLI-27
(AICTE approved, Affiliated to VTU and ISO 9001:2015 Certified Institute)
DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING
(NBA Accredited Program)



Opposite Airport, Gokul, Hubballi-580 027. Dist.: Dharwad, Karnataka, India.
Phone: 0836-2232 664, 681 Fax: 0836-2330688, E-mail: hod_ece@kleit.ac.in, website: www.kleit.ac.in

Laboratory Manual

for

IV SEMESTER

PRINCIPALS OF COMMUNICATION SYSTEMS LABORATORY MANUAL

21ECL46

Prepared by: Mr. Kotresk B. K. & Miss. Supriya Puttappanavar

Approved by: Dr. G. A. Bidkar

Vision and Mission of the Department

Vision

Prepare professionals embodied with competence and values to meet the challenges in the field of Electronics and Communication Engineering.

Mission

- m1. Providing hands-on experience with state-of-the-art hardware and software infrastructure, enhancing competency of committed faculty to meet the requirements of industry and research institutions.
- m2. Inculcating theoretical and practical foundations and design and development skills necessary for lifelong learning.
- m3. Organizing value addition courses and continuous industry-institute interactions for bridging the gap between academia and industry.
- m4. Imbibing human values and ethics to make them socially responsible professionals.

Program Educational Objectives (PEOs)

- 1. Graduates of the program will be able to apply the knowledge of Basic Sciences, mathematics and engineering fundamentals to provide solutions to the problems in the field of Electronics and Communication.
- 2. Graduates of the program will be able to choose professional career in electronics and communication fields, research, entrepreneurship, academia, and engineering and administrative services.
- 3. Graduates of the program will be able to design, install and commission, operate and maintain electronics and communication systems.
- 4. Graduates of the program will be able to demonstrate interpersonal skills and lifelong learning attitude in their profession.
- 5. Graduates of the program will be able to address and solve ethically the issues related to profession, society and environment.

Program Outcomes (POs)

Engineering Graduates will be able to:

1. **Engineering knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
2. **Problem analysis:** Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
3. **Design/development of solutions:** Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
4. **Conduct investigations of complex problems:** Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
5. **Modern tool usage:** Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.
6. **The engineer and society:** Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
7. **Environment and sustainability:** Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
8. **Ethics:** Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
9. **Individual and teamwork:** Function effectively as an individual, and as a member or leader in diverse

teams, and in multidisciplinary settings.

10. **Communication:** Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
11. **Project management and finance:** Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
12. **Life-long learning:** Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

Program Specific Outcomes (PSOs)

PSO.1: Analyze, design, build and test analog, digital communication and embedded systems for given specifications.

Pre-requisites:

- i. Fundamentals of Analog Modulation Techniques
- ii. Fundamentals of Digital Modulation Techniques

Course Outcomes-CO

1. Understand and analyse concepts of Analog Modulation schemes viz; AM, FM
2. Design and analyse the electronic circuits for AM and FM modulation and demodulation.
3. Understand the concepts of random variable and random process to model communication systems.
4. Understand and analyse the concepts of digitization of signals.
5. Evolve the concept of SNR in the presence of channel induced noise

CO-PO Mapping

Course Outcomes-CO	1	2	3	4	5	6	7	8	9	10	11	12	PSO1
	Engineering knowledge	Problem analysis	Design/development of solutions	Conduct investigations of complex problems	Modern tool usage	The engineer and society	Environment and sustainability	Ethics	Individual and team work	Communication	Project management and finance	Life-long learning	Analyze, design, build and test analog, digital communication and embedded systems for given specifications.
Understand and analyse concepts of Analog Modulation schemes viz; AM, FM.	1	1	2	-	1	-	-	2	3	3	-	1	3
Design and analyse the electronic circuits for AM and FM modulation and demodulation	2	3	2	-	1	-	-	2	3	3	-	1	3
Understand the concepts of random variable and random process to model communication systems.	2	1	2	-	1	-	-	2	3	3	-	1	3
Understand and analyse the concepts of digitization of signals.	2	1	2	-	1	-	-	2	3	3	-	1	3

Evolve the concept of SNR in the presence of channel induced noise	2	1	2	-	1	-	-	2	3	3	-	1	3
Overall	2	2	2	-	1	-	-	2	3	3	-	1	3

VTUSYNC.IN

Course Content

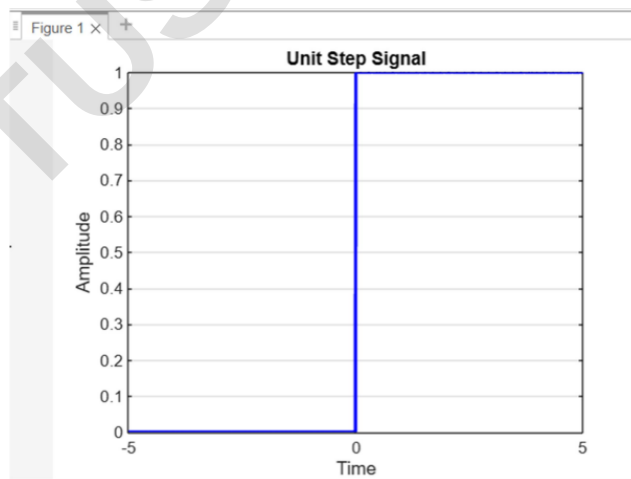
Si. No.	Title
1	Basic Signals and Signal Graphing: a) unit Step, b) Rectangular, c) standard triangle d) sinusoidal and e)Exponential signal.
2	Illustration of signal representation in time and frequency domains for a rectangular pulse.
3	Amplitude Modulation and demodulation: Generation and display the relevant signals and its spectrums.
4	Frequency Modulation and demodulation: Generation and display the relevant signals and its spectrums.
5	Sampling and reconstruction of low pass signals. Display the signals and its spectrum.
6	Time Division Multiplexing and demultiplexing.
7	PCM Illustration: Sampling, Quantization and Encoding
8	Generate a)NRZ, RZ and Raised cosine pulse, b) Generate and plot eye diagram
9	Generate the Probability density function of Gaussian distribution function.
10	Display the signal and its spectrum of an audio signal.

Si. No.	Title
1	Basic Signals and Signal Graphing: a) unit Step, b) Rectangular, c) standard triangle d) sinusoidal and e)Exponential signal.

Program:

a) Unit step

```
% Define the time range
t = -5:0.01:5;
% Define the unit step signal function
unit_step = @(t) (t >= 0);
% Generate the unit step signal
unit_step_signal = unit_step(t);
% Plotting
plot(t, unit_step_signal, 'b', 'LineWidth', 2);
title('Unit Step Signal');
xlabel('Time');
ylabel('Amplitude');
grid on;
```

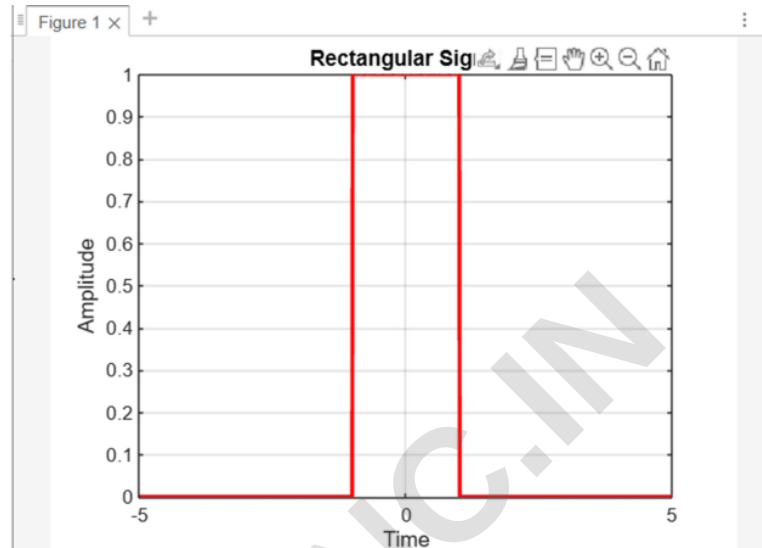


b) Rectangular

```
% Define the time range
t = -5:0.01:5;
% Define the rectangular signal function
rectangular = @(t, width) (abs(t) <= width/2);
% Set the width of the rectangular signal
width_rectangular = 2;
% Generate the rectangular signal
rectangular_signal = rectangular(t, width_rectangular);
```

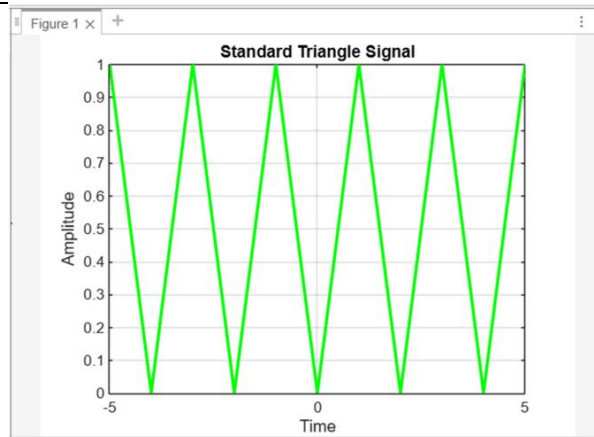
----- Studied smart, not hard — thanks to VTUSync.in -----


```
% Plotting
plot(t, rectangular_signal, 'r', 'LineWidth', 2);
title('Rectangular Signal');
xlabel('Time');
ylabel('Amplitude');
grid on;
```



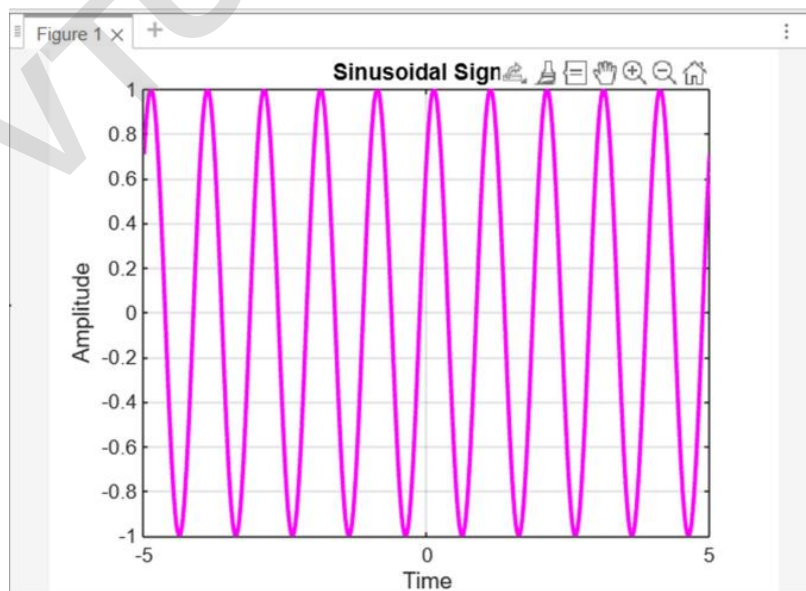
c) Standard Triangle

```
% Define the time range
t = -5:0.01:5;
% Define the standard triangle signal function
standard_triangle = @(t, period) (1 - abs(mod(t, period) -
period/2) / (period/2));
% Set the period of the triangle signal
period_triangle = 2;
% Generate the standard triangle signal
standard_triangle_signal = standard_triangle(t, period_triangle);
% Plotting
plot(t, standard_triangle_signal, 'g', 'LineWidth', 2);
title('Standard Triangle Signal');
xlabel('Time');
ylabel('Amplitude');
grid on;
```



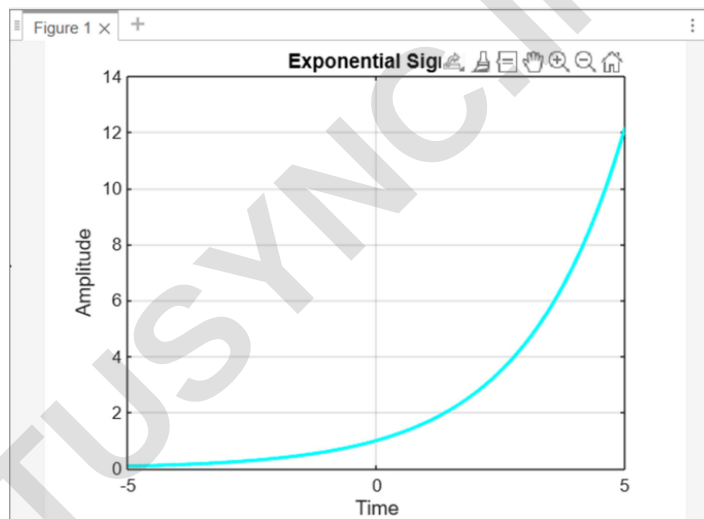
d) Sinusoidal

```
% Define the time range
t = -5:0.01:5;
% Define the parameters for the sinusoidal signal
frequency_sine = 1; % Frequency in Hz
phase_sine = pi/4; % Phase shift in radians
% Generate the sinusoidal signal
sinusoidal_signal = sin(2*pi*frequency_sine*t + phase_sine);
% Plotting
plot(t, sinusoidal_signal, 'm', 'LineWidth', 2);
title('Sinusoidal Signal');
xlabel('Time');
ylabel('Amplitude');
grid on;
```



e) Exponential signal

```
% Define the time range
t = -5:0.01:5;
% Define the parameters for the exponential signal
alpha_exp = 0.5; % Decay rate
% Generate the exponential signal
exponential_signal = exp(alpha_exp * t);
% Plotting
plot(t, exponential_signal, 'c', 'LineWidth', 2);
title('Exponential Signal');
xlabel('Time');
ylabel('Amplitude');
grid on;
```

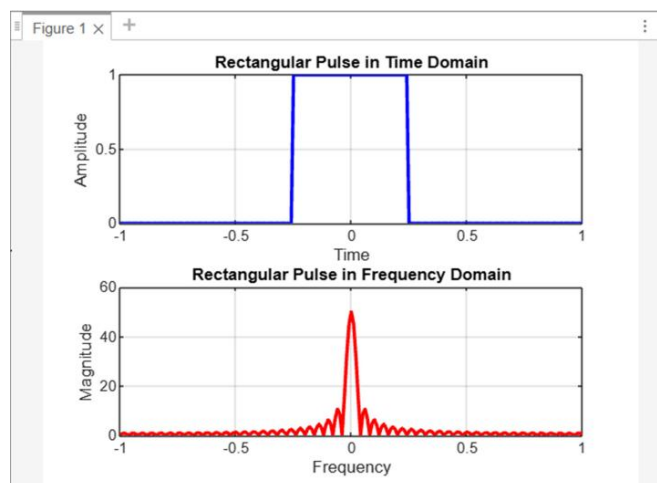


Si. No.	Title
2	Illustration of signal representation in time and frequency domains for a rectangular pulse.

Program:

```
% Define time vector
t = -1:0.01:1;
% Generate rectangular pulse
rect_pulse = rectpuls(t, 0.5); % Rectangular pulse with width 0.5
units
% Compute Fourier Transform
fft_pulse = fftshift(fft(rect_pulse));
frequencies = linspace(-1, 1, length(fft_pulse));
% Plotting
figure;
% Time Domain Plot
subplot(2,1,1);
plot(t, rect_pulse, 'b', 'LineWidth', 2);
title('Rectangular Pulse in Time Domain');
xlabel('Time');
ylabel('Amplitude');
grid on;
% Frequency Domain Plot
subplot(2,1,2);
plot(frequencies, abs(fft_pulse), 'r', 'LineWidth', 2);
title('Rectangular Pulse in Frequency Domain');
xlabel('Frequency');
ylabel('Magnitude');
grid on;
```

Result:



Si. No.	Title
3	Amplitude Modulation and demodulation: Generation and display the relevant signals and its spectrums.

Program:

```
Fs = 1000; % Sampling frequency (Hz)
T = 1/Fs; % Sampling period
L = 1000; % Length of signal
t = (0:L-1)*T; % Time vector
Fc = 100; % Carrier frequency (Hz)
Am = 0.5; % Modulation depth (amplitude of modulating signal)
Fm = 10; % Modulating frequency (Hz)

% Modulating signal (message signal)
m = Am*sin(pi*Fm*t);

% Carrier signal
c = sin(pi*Fc*t);

% Modulated signal (AM signal)
s = (1 + m).*c;

% Demodulation
s_demod = s ./ c; % Demodulation by multiplying with carrier

% Plotting
figure;

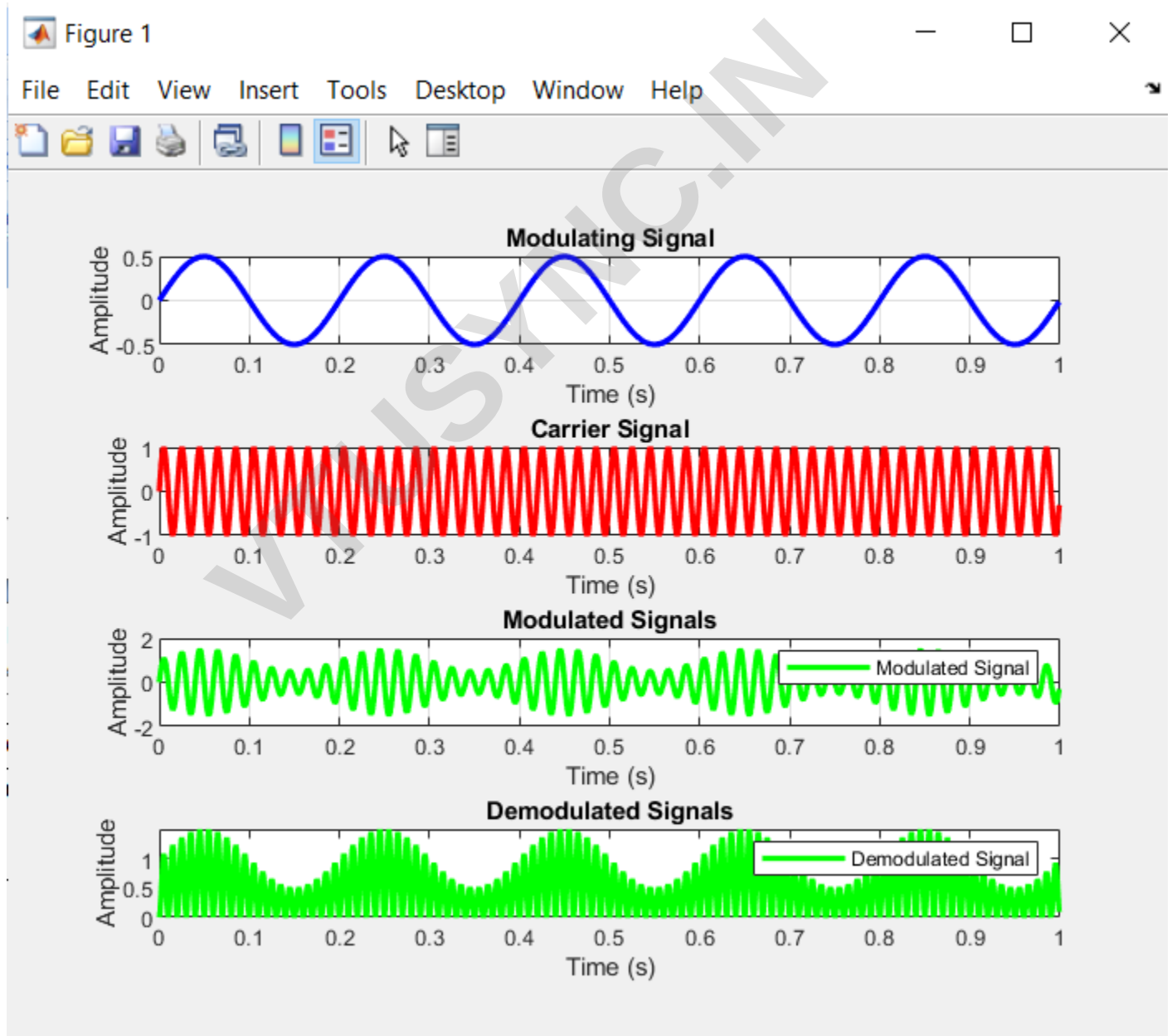
% Time Domain Plots
subplot(4, 1, 1);
plot(t, m, 'b', 'LineWidth', 2);
title('Modulating Signal');
xlabel('Time (s)');
ylabel('Amplitude');
grid on;

subplot(4, 1, 2);
plot(t, c, 'r', 'LineWidth', 2);
title('Carrier Signal');
xlabel('Time (s)');
ylabel('Amplitude');
grid on;

subplot(4, 1, 3);
plot(t, s, 'g', 'LineWidth', 2);
title('Modulated Signals');
xlabel('Time (s)');
```

```
ylabel('Amplitude');  
legend('Modulated Signal');  
grid on;  
  
subplot(4, 1, 4);  
plot(t, s_demod, 'g', 'LineWidth', 2);  
title('Demodulated Signals');  
xlabel('Time (s)');  
ylabel('Amplitude');  
legend('Demodulated Signal');
```

Result:



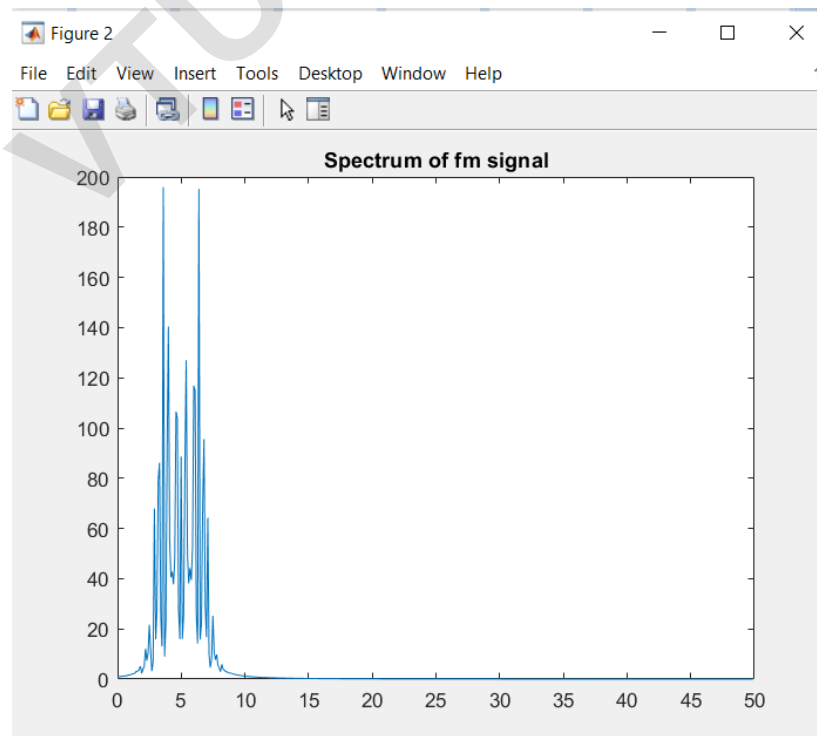
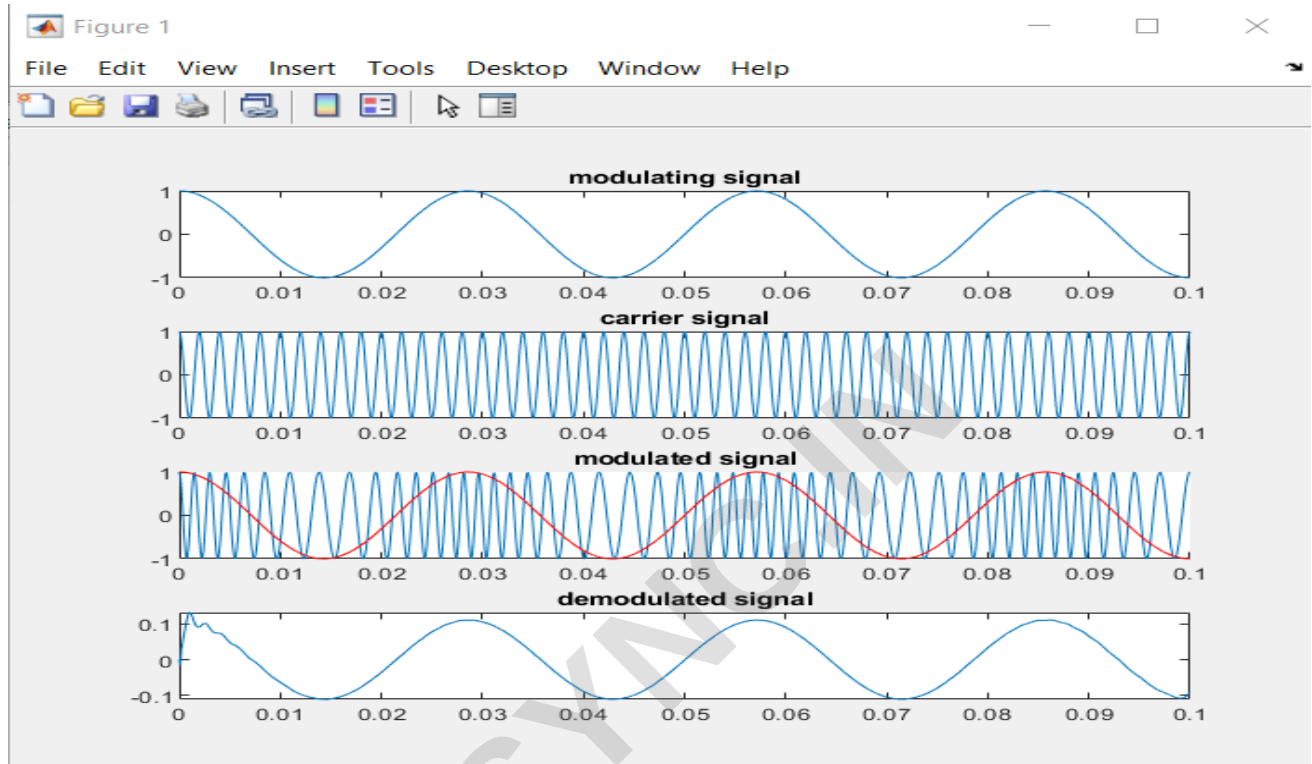
Si. No.	Title
4	Frequency Modulation and demodulation: Generation and display the relevant signals and its spectrums.

Program:

```
fs=10000;
ac=1;
am=1;
fm=35;
fc=500;
b=5;
t=(0:0.1*fs)/fs;
wc=2*pi*fc;
wm=2*pi*fm;
mt=am*cos(wm*t);
subplot(4,1,1);
plot(t,mt);
title('modulating signal');
ct=ac*cos(wc*t);
subplot(4,1,2);
plot(t,ct);
title('carrier signal');
st=ac*cos((wc*t)+b*sin(wm*t));
subplot(4,1,3);
hold on;
plot(t,st);
plot(t,mt,'r');
hold off;
title('modulated signal');
d=demod(st,fc,fs,'fm');
subplot(4,1,4);
plot(t,d);
title('demodulated signal');
Fs = 100;
%sampling frq
t = (0:2*Fs+1)/Fs;
Fc = 10; % Carrier frequency
%x = sin(2*pi*2*t); % message signal
Ac=1;
% spectrum of fm
%xfm=fmmod(x,Fc,Fs,10);
zfm = fft(st);
zfm = abs(zfm(1:length(zfm)/2+1));
frqfm = (0:length(zfm)-1)*Fs/length(zfm)/2;
figure(2);
```

```
plot(frqfm,zfm);  
title('Spectrum of fm signal');
```

Result:



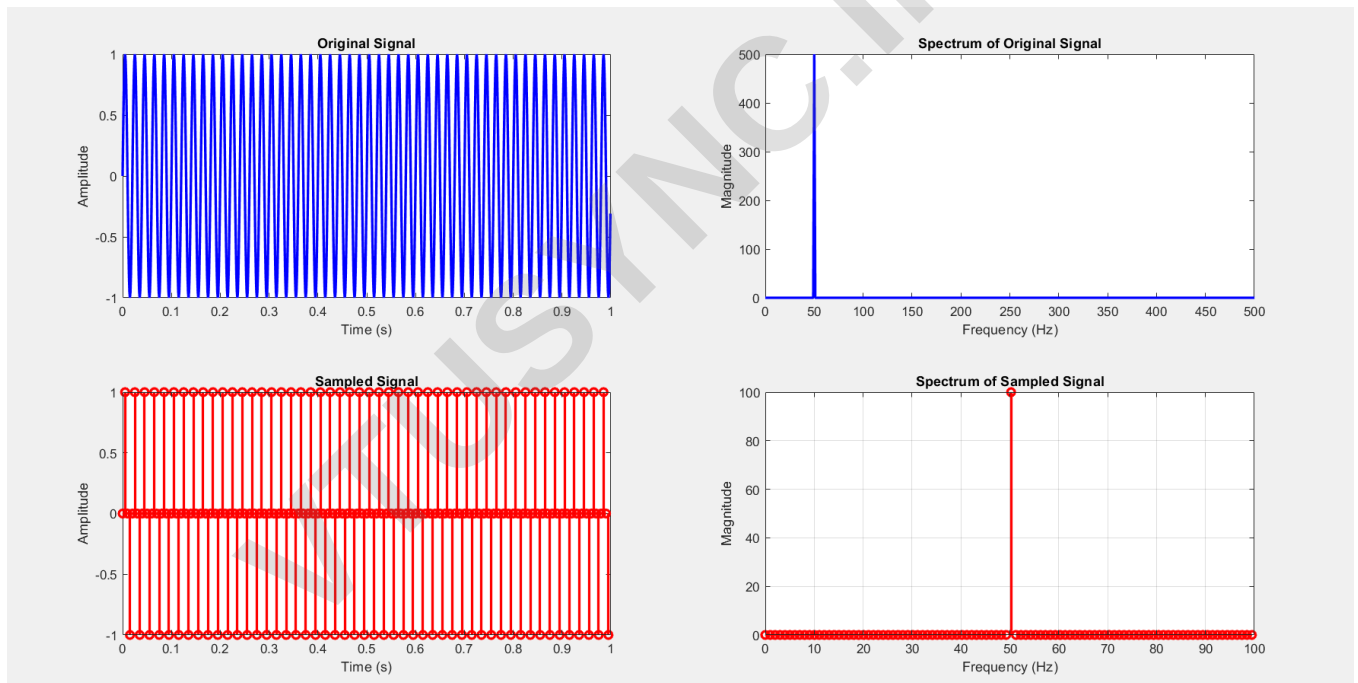
Si. No.	Title
5	Sampling and reconstruction of low pass signals. Display the signals and its spectrum.

Program:

```
% Sampling and Reconstruction of Low Pass Signals
clc;
clear all;
close all;
% Parameters
Fs = 1000; % Sampling frequency (Hz)
T = 1/Fs; % Sampling period
duration = 1; % Duration of signal (seconds)
f_signal = 50; % Signal frequency (Hz)
A_signal = 1; % Signal amplitude
% Time vector
t = 0:T:duration-T;
% Generate low-pass signal
x = A_signal * sin(2*pi*f_signal*t);
% Plot original signal and its spectrum
figure;
subplot(2, 2, 1);
plot(t, x, 'b', 'LineWidth', 2);
title('Original Signal');
xlabel('Time (s)');
ylabel('Amplitude');
% Compute and plot spectrum of original signal
X = fft(x);
frequencies = linspace(0, Fs, length(X));
subplot(2, 2, 2);
plot(frequencies, abs(X), 'b', 'LineWidth', 2);
title('Spectrum of Original Signal');
xlabel('Frequency (Hz)');
ylabel('Magnitude');
xlim([0, Fs/2]);
% Sampling
Fs_new = 200; % New sampling frequency (Hz)
T_new = 1/Fs_new; % New sampling period
t_new = 0:T_new:duration-T_new;
x_sampled = A_signal * sin(2*pi*f_signal*t_new);
% Plot sampled signal and its spectrum
subplot(2, 2, 3);
stem(t_new, x_sampled, 'r', 'LineWidth', 2);
title('Sampled Signal');
xlabel('Time (s)');
```

```
ylabel('Amplitude');  
% Compute and plot spectrum of sampled signal  
X_sampled = fft(x_sampled);  
frequencies_sampled = linspace(0, Fs_new, length(X_sampled));  
subplot(2, 2, 4);  
stem(frequencies_sampled, abs(X_sampled), 'r', 'LineWidth', 2);  
title('Spectrum of Sampled Signal');  
xlabel('Frequency (Hz)');  
ylabel('Magnitude');  
xlim([0, Fs_new/2]);  
% Adjust plot  
grid on;
```

Result:



Si. No.	Title
6	Time Division Multiplexing and Demultiplexing.

Program:

```
% Number of signals to multiplex
numSignals = 3
% Number of time slots
numSlots = 10
% Generate random data for each signal
data = cell(1, numSignals);
for i = 1:numSignals
    data{i} = randi([0, 100], 1, numSlots) % Generate random data for
each signal
end
% Multiplexing (combining signals into one)
TDM_signal = zeros(1, numSlots*numSignals)
for i = 1:numSignals
    TDM_signal((i-1)*numSlots+1:i*numSlots) = data{i}
end
% Display multiplexed signal
disp('Multiplexed Signal (TDM):');
disp(TDM_signal);
% Demultiplexing (separating signals)
demux_data = cell(1, numSignals);
for i = 1:numSignals
    demux_data{i} = TDM_signal(i:numSignals:end); % Extract every nth
element
end
% Display demultiplexed signals
for i = 1:numSignals
    fprintf('Demultiplexed Signal %d:\n', i);
    disp(demux_data{i});
end
```

Result:

Multiplexed Signal (TDM):

Columns 1 through 18

13 21 18 4 10 62 94 35 41 99 95 68 99 77 34 66 24 29

Columns 19 through 30

68 53 41 60 75 58 55 58 51 8 72 100

Demultiplexed Signal 1:

13 4 94 99 99 66 68 60 55 8

Demultiplexed Signal 2:

21 10 35 95 77 24 53 75 58 72

Demultiplexed Signal 3:

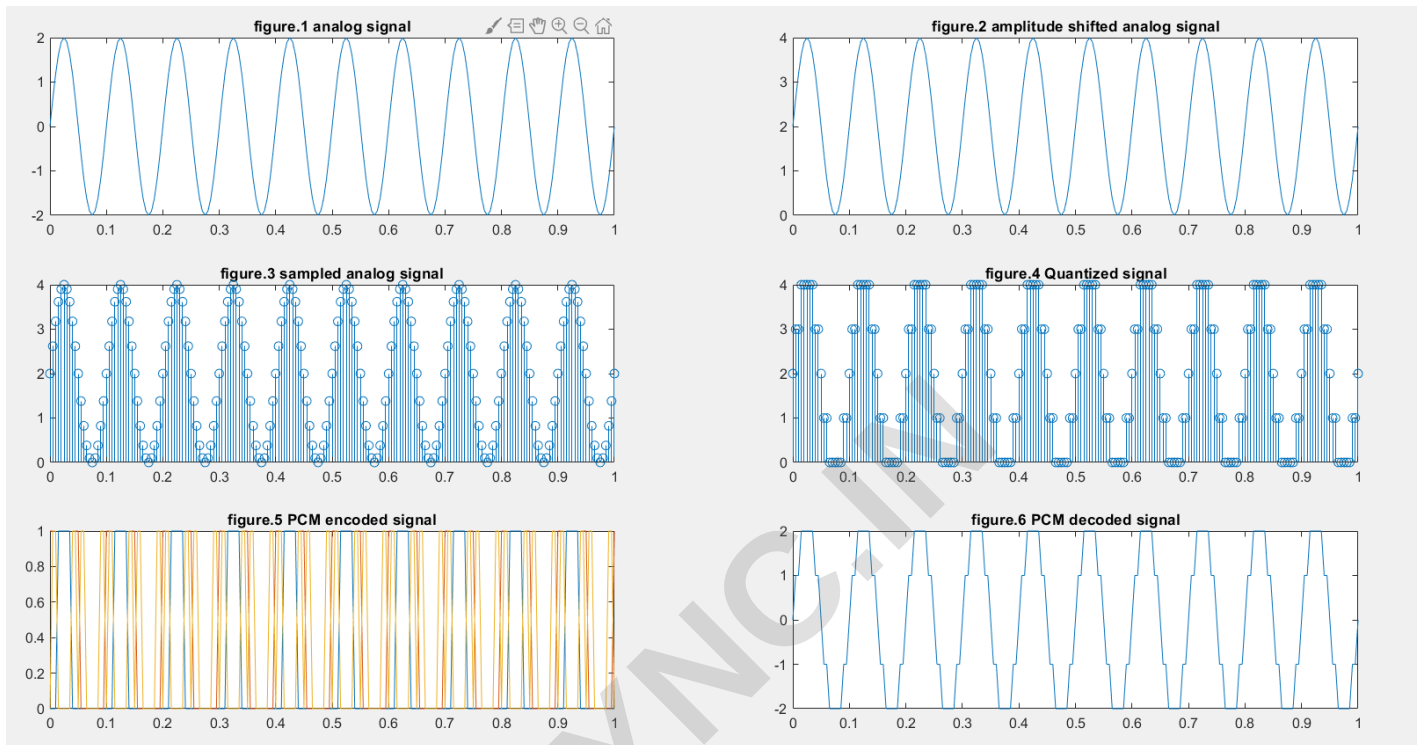
18 62 41 68 34 29 41 58 51 100

Si. No.	Title
7	PCM Illustration: Sampling, Quantization and Encoding

Program:

```
%PCM Encoding clc;
clear all; close all;
f=10;
fs=20*f; t=0:1/fs:1; a=2;
x=a*sin(2*pi*f*t); subplot(321); plot(t,x);
title('figure.1 analog signal');
x1=x+2;
subplot(322); plot(t,x1);
title('figure.2 amplitude shifted analog signal');
subplot(323); stem(t,x1);
title('figure.3 sampled analog signal');
q=round(x1); subplot(324); stem(t,q);
title('figure.4 Quantized signal');
enco=de2bi(q,'left-msb'); subplot(325); plot(t,enco);
title('figure.5 PCM encoded signal');
%PCM Decoding
deco=bi2de(enco,'left-msb');
deco=deco-2;
subplot(326); plot(t,deco);
title('figure.6 PCM decoded signal');
```

Result:



Si. No.	Title
8	Generate a)NRZ, RZ and Raised cosine pulse, b) Generate and plot eye diagram

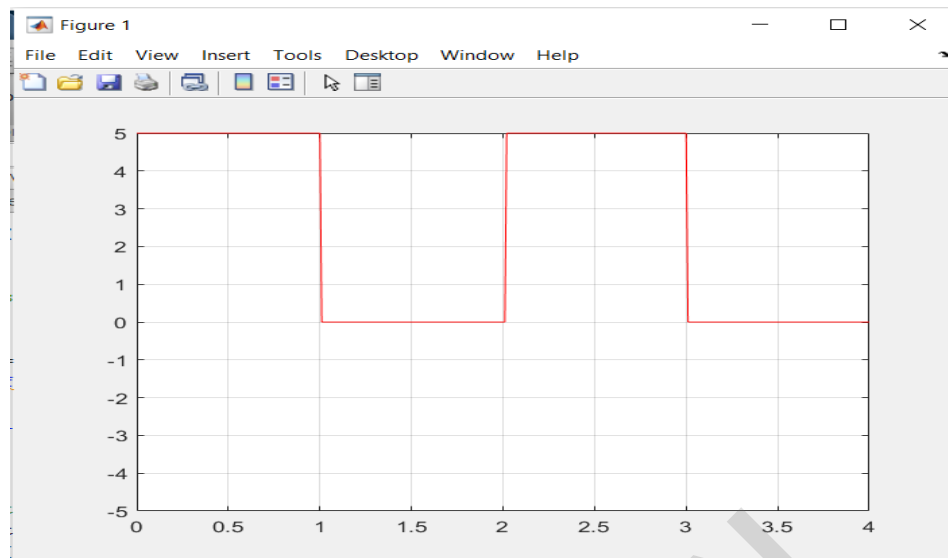
Program:

a)NRZ, RZ and Raised cosine pulse

- Unipolar NRZ

```
%unipolar nrz line coding

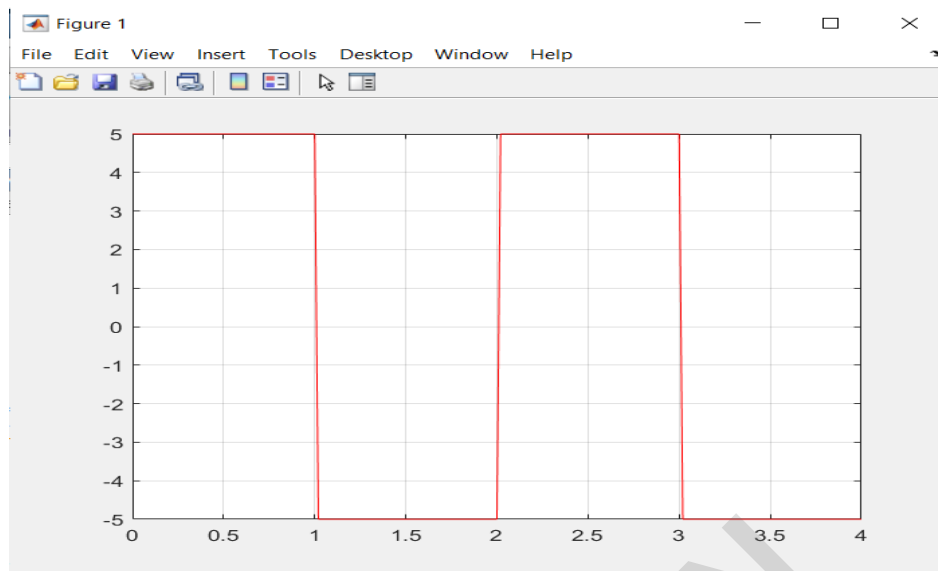
clc;
clear all;
close all;
warning off;
n=input('enter the input sequence');
%n=[1,0,1,0];
% Mapping of bits to voltage levels 5v and 0v
for i=1:length(n)
if n(i)==1
    m(i)=5;
else
    m(i)=0;
end
end
% pulse shaping
i=1
t=0:0.01:length(n);
for j=1:length(t)
    if t(j)<=i;
        y(j)=m(i);
    else
        i=i+1;
    end
end
end
% plotting pulse
plot(t,y,'r-');
axis([0 length(n) -5 5]);
grid on;
```



- **Polar NRZ**

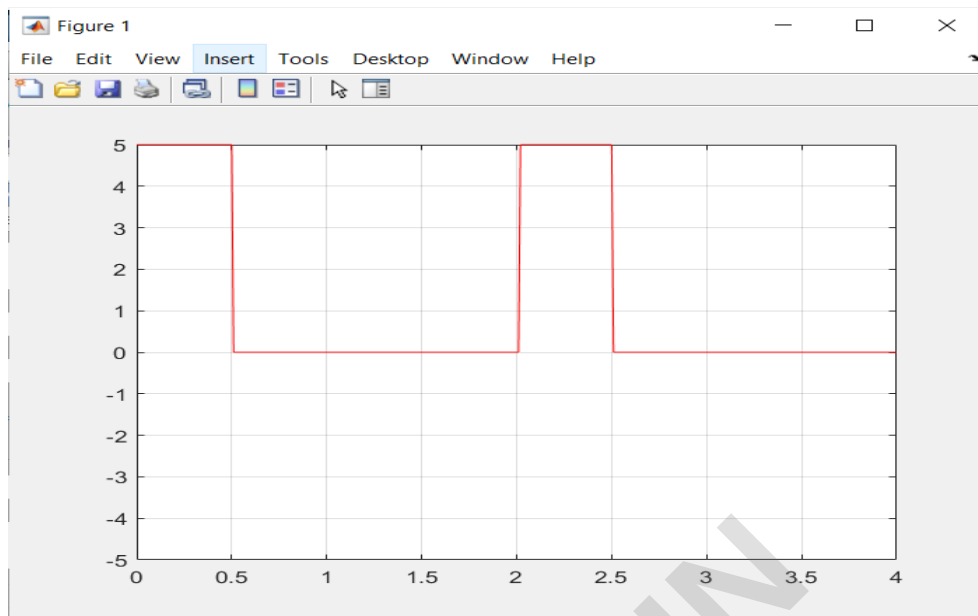
%unipolar nrz line coding

```
clc;
clear all;
close all;
warning off;
n=input('enter the input sequence');
%n=[1,0,1,0];
% Mapping of bits to voltage levels 5v and 0v
for i=1:length(n)
if n(i)==1
    m(i)=5;
else
    m(i)=-5;
end
end
% pulse shaping
i=1
t=0:0.01:length(n);
for j=1:length(t)
    if t(j)<=i;
        y(j)=m(i);
    else
        i=i+1;
    end
end
% plotting pulse
plot(t,y,'r-');
axis([0 length(n) -5 5]);
grid on;
```

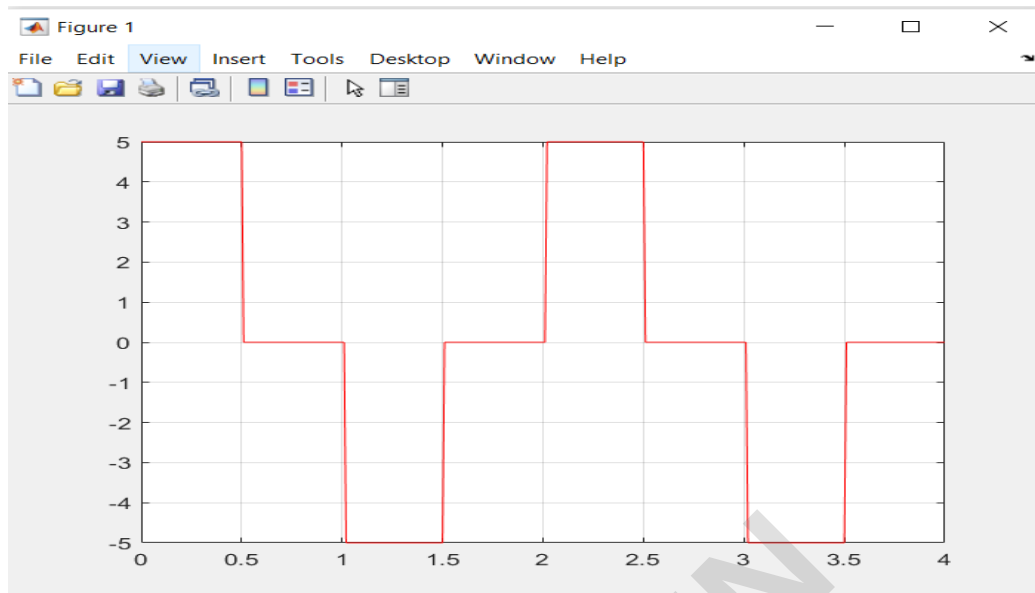
- **Unipolar RZ**

```
% unipolar rz line coding
clc;
clear all;
close all;
%n=input('enter the input sequence');
n=[1,0,1,0];
% Mapping of bits to voltage levels 5v and 0v
for i=1:length(n)
if n(i)==1;
m(i)=5;
else
m(i)=0;
end
end
% pulse shaping
a=0;
b=0.5;
i=1;
t=0:0.01:length(n);
for j=1:length(t)
if t(j)>=a && t(j)<=b ; y(j)=m(i);
elseif t(j)>b && t(j)<=i y(j)=0;
else
i=i+1; a=a+1; b=b+1;
end
end
% plotting pulse
plot(t,y,'r-');
grid on;
axis([0 length(n) -5 5]);
```



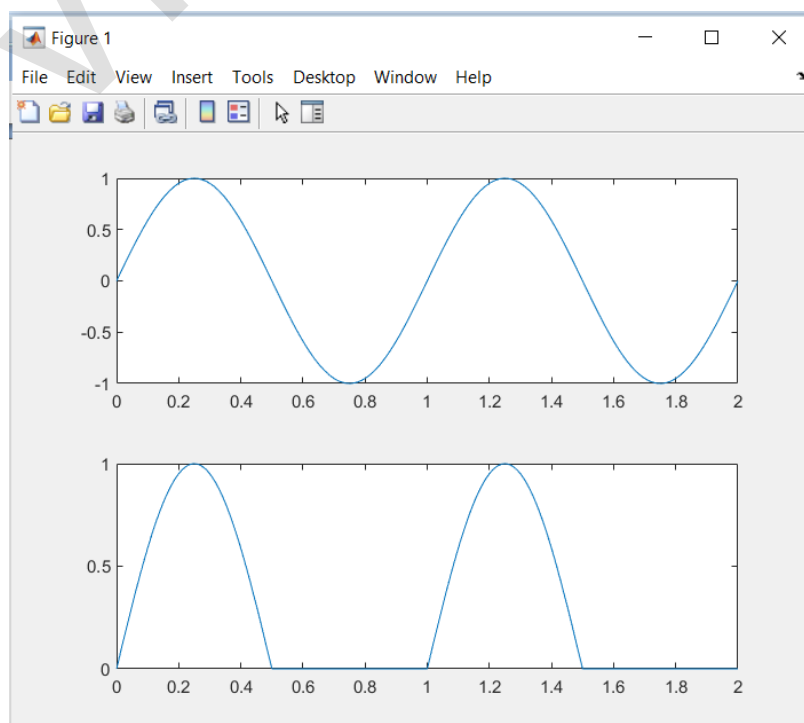
- **Polar RZ**

```
% unipolar rz line coding
clc;
clear all;
close all;
%n=input('enter the input sequence');
n=[1,0,1,0];
% Mapping of bits to voltage levels 5v and 0v
for i=1:length(n)
if n(i)==1;
m(i)=5;
else
m(i)=-5;
end
end
% pulse shaping
a=0;
b=0.5;
i=1;
t=0:0.01:length(n);
for j=1:length(t)
if t(j)>=a && t(j)<=b ; y(j)=m(i);
elseif t(j)>b && t(j)<=i y(j)=0;
else
i=i+1; a=a+1; b=b+1;
end
end
% plotting pulse
plot(t,y,'r-');
grid on;
axis([0 length(n) -5 5]);
```



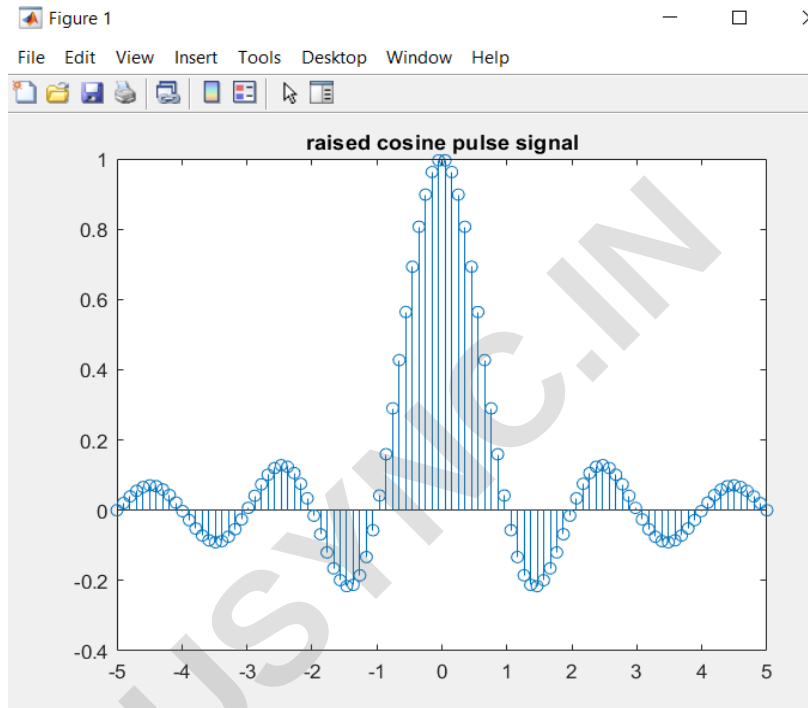
- **Half Sinusoidal Signal**

```
% Half Sinusoid
clc;
clear all; close all;
t = 0:0.01:2;
f=1;% Input Signal Frequency x=sin(2*pi*f*t);%
Generate Sine Wave subplot(211);
plot(t,x);
x(x<0) = 0; % Rectified Sine Wave
subplot(212);
plot(t,x);
```



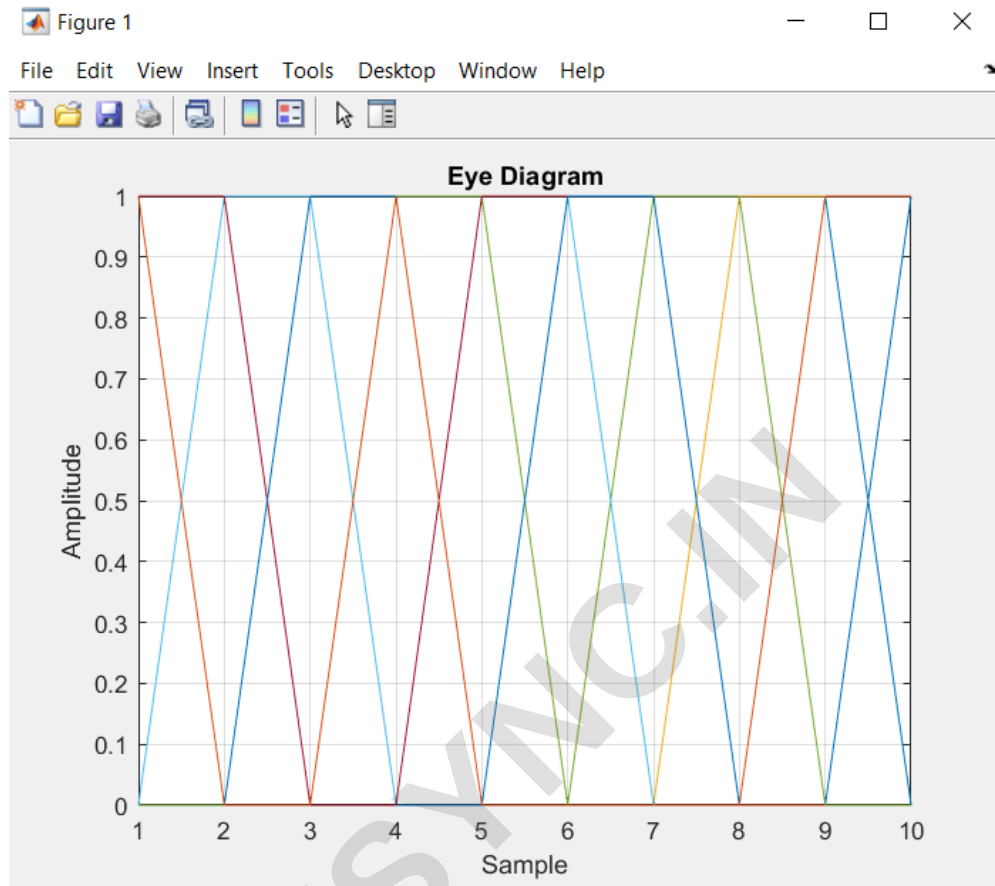
- **Raised cosine signal**

```
% raised cosin pulseclc;  
clear all;close all;  
% warning off;  
x=linspace(-5,5);  
y=sinc(x); stem(x,y);  
title('raised cosine pulse signal');
```



b) Generate and plot eye diagram

```
% Generate a random binary signal  
data_length = 1000;  
data = randi([0, 1], 1, data_length);  
% Reshape the data into a matrix with desired number of samples  
per symbol  
samples_per_symbol = 10;  
num_symbols = data_length / samples_per_symbol;  
eye_data = reshape(data, samples_per_symbol, num_symbols);  
% Plot the eye diagram  
figure;  
plot(eye_data);  
title('Eye Diagram');  
xlabel('Sample');  
ylabel('Amplitude');  
grid on;
```

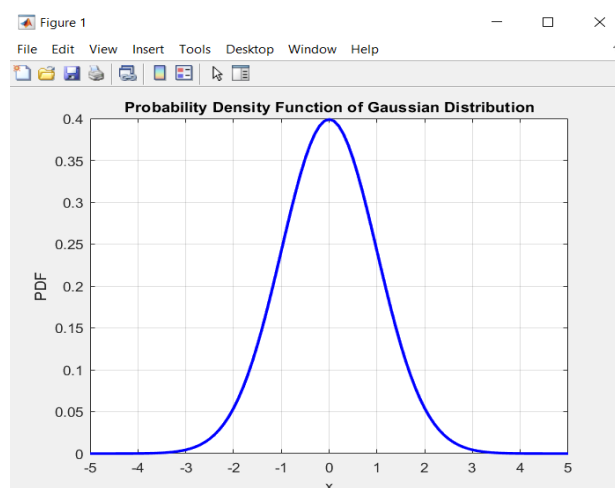


Si. No.	Title
9	Generate the Probability density function of Gaussian distribution function.

Program:

```
% Parameters
mu = 0; % Mean of the Gaussian distribution
sigma = 1; % Standard deviation of the Gaussian distribution
% Define range for x-axis
x = -5:0.1:5;
% Compute PDF
pdf = (1 / (sigma * sqrt(2 * pi))) * exp(-(x - mu).^2 / (2 *
sigma^2));
% Plot PDF
figure;
plot(x, pdf, 'b', 'LineWidth', 2);
title('Probability Density Function of Gaussian Distribution');
xlabel('x');
ylabel('PDF');
grid on;
```

Result:



Si. No.	Title
10	Display the signal and its spectrum of an audio signal.

Program:

```
% Load audio file
audioFile = 'your_audio_file.wav'; % Change this to your audio
file path
[y, Fs] = audioread(audioFile);
% Plot the time-domain signal
t = (0:length(y)-1) / Fs;
figure;
subplot(2, 1, 1);
plot(t, y);
title('Time-Domain Signal');
xlabel('Time (s)');
ylabel('Amplitude');
% Compute the spectrum
N = length(y);
Y = fft(y);
f = (0:N-1) * Fs / N;
% Plot the spectrum
subplot(2, 1, 2);
plot(f, abs(Y));
title('Frequency Spectrum');
xlabel('Frequency (Hz)');
ylabel('Magnitude');
xlim([0, Fs/2]); % Display only positive frequencies
% You can also use the following line to display the spectrum in
dB scale
% plot(f, 20*log10(abs(Y)));
% ylabel('Magnitude (dB)');
% Adjust plot
grid on
```

Result:

